

Maestro

Agile SDD com TDD

Framework de orquestração de desenvolvimento
com múltiplos agentes de IA

Specification-Driven · Test-Driven · Human-in-the-Loop

Quem Sou Eu

Esta apresentação ensina qualquer pessoa — com ou sem conhecimento técnico — a usar o Maestro Agile SDD com TDD para desenvolver software com qualidade garantida.

O que você vai aprender

1. O problema que o Maestro resolve
2. Os 5 papéis: PO, Maestro, BA, Dev, QA
3. Os 5 valores e 7 princípios
4. SDD + TDD: especificação E testes antes do código
5. Os 6 Quality Gates e o ciclo TDD (RED → GREEN → REFACTOR)
6. Human-in-the-Loop: você sempre aprova
7. Fura-fila com detecção proativa
8. Como começar um projeto do zero

O Problema Que o Maestro Resolve

Sem Orquestração ✗

Caos de requisitos

Features sem especificação. Ninguém sabe o que entregar.

Testes depois do código

"Funcionou na minha máquina." Testes viram obrigação burocrática, não ferramenta de design.

Zero rastreabilidade

Impossível saber qual requisito originou qual código.

Memória zero

Cada sessão começa do zero.

Com Maestro Agile SDD + TDD ✓

Specification-First

Toda feature nasce como Gherkin formal. Spec é contrato.

Test-Driven Development

Testes são escritos ANTES do código. ● RED → ● GREEN → ● REFACTOR.

Rastreabilidade total

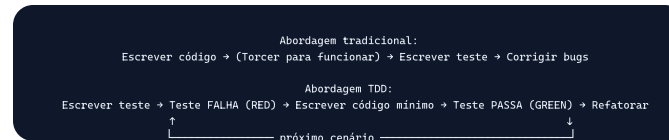
Spec → Teste → Código. Tudo linkado.

Memória persistente

`memory/session-latest.md` registra cada iteração.

Por Que TDD?

Test-Driven Development: escrever o teste ANTES do código



TDD garante:

- ☒ Código só existe se há teste que o justifique
- ☒ Zero código desnecessário (YAGNI)
- ☒ Design guiado por uso real, não por especulação
- ☒ Refatoração segura (testes pegam regressão)
- ☒ Cobertura de testes naturalmente alta

Sem TDD:

- ☒ Testes escritos depois viram viés de confirmação
- ☒ Código testa o código, não o requisito
- ☒ "Funciona" sem saber se é o que deveria funcionar
- ☒ Refatoração é arriscada (sem rede de segurança)

O Documento Canônico

SOURCE_OF_TRUTH.md é o documento mais importante do projeto. Em caso de conflito entre qualquer artefato, ele vence. Sempre.

O que contém

Seção	Conteúdo
1. Identidade	Nome, pitch, stack, idioma, público-alvo
2. Metodologia	Maestro Agile SDD com TDD, valores, princípios, ciclo
3. Papéis	Product Owner, Maestro, BA, Dev, QA — funções e fases
4. Governance	HITL obrigatório, 6 Quality Gates, protocolo de aprovação
5. Processo	6 fases com TDD, entradas, saídas e gates
6. Fura-fila	Deteção proativa, protocolo, restrições
7. DoD	Definition of Done — checklist para milestone completo
8. Roadmap	Milestones mapeados com prioridades e status
9. Referências	Maestro - Agile SDD com TDD Links para todos os artefatos complementares

Os 5 Papéis

Papel	Quem	Fases	Responsabilidade
 Product Owner	Você (humano)	G0, G1, G5	Visão do produto, prioridades, aprova gates manuais
 Maestro	Claude (IA)	Todas	Orquestra o ciclo, garante gates, detecta fura-fila, mantém memória
 Business Analyst	Agente IA	FASE 0	Transforma milestones em especificações Gherkin formais
 Full Stack Dev	Agente IA	FASE 1 + FASE 3	Deriva arquitetura da spec + Implementa código que faz testes passarem (TDD)
 QA Engineer	Agente IA	FASE 2 + FASE 4	Escreve testes ANTES do código (RED) + Confirma deploy preview

Regra de comunicação: Product Owner ↔ Maestro ↔ Agentes. Nunca se pula hierarquia.

Os 5 Valores



Specification First

Nada é implementado sem spec Gherkin aprovada. A spec é o contrato.



Test First (TDD)

Testes são escritos ANTES do código. ● RED → ● GREEN → ● REFACTOR. Código sem teste é código que não deveria existir.



Human in the Loop

Toda decisão de avanço é bloqueada até um humano liberar. Automação informa; humano decide.



Quality Non-Negotiable

Nenhum gate pode ser bypassado. Gate fechado = STOP.



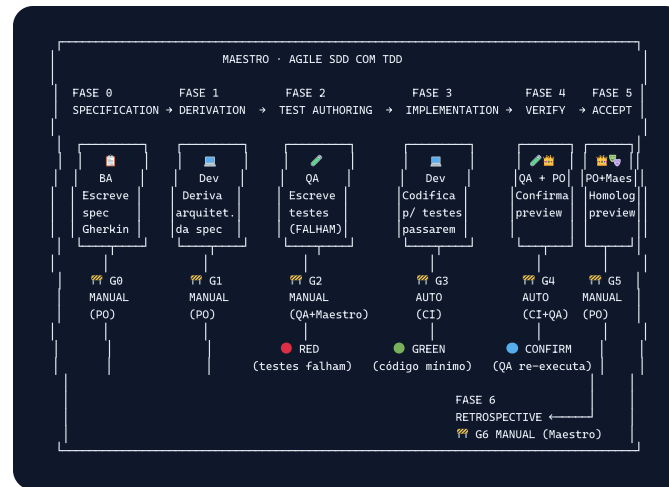
Continuous Improvement

Cada ciclo termina com uma melhoria concreta no processo. O framework evolui a cada sprint.

Os 7 Princípios

#	Princípio	Na prática
P1	Spec é contrato	O que está na spec Gherkin É o que será implementado e testado
P2	Testes primeiro	Teste existe antes do código. Se não tem teste, não tem código
P3	Gate fechado = STOP	Nenhum agente avança com gate atual fechado
P4	Uma fonte da verdade	<code>SOURCE_OF_TRUTH.md</code> é canônico; todo outro documento é subordinado
P5	Approval é humano	Build verde e testes passando informam, mas não decidem por você
P6	Falhe rápido	Erro na FASE 0 custa 10x menos que na FASE 3
P7	Fura-fila é detectado	Maestro identifica desvios proativamente; PO não precisa saber o termo

O Ciclo SDD com TDD



2-3)



RED

FASE 2

QA escreve teste

Teste FALHA



GREEN

FASE 3

Dev escreve código
mínimo para passar



REFACTOR

FASE 3

Dev otimiza código

Teste continua

Os 6 Quality Gates

Gate	Nome	Fase	Verifica	Quem	Tipo
G0	Spec Approved	FASE 0	Gherkin 100% completo, zero ambiguidades	👑 PO	MANUAL
G1	Architecture Approved	FASE 1	Arquitetura cobre toda a spec, decisões justificadas	👑 PO	MANUAL
G2	Tests Authored (RED)	FASE 2	Testes E2E+unitários escritos, cobrem 100% spec, e FALHAM	🧪 QA + 🎭	MANUAL
G3	All Tests Pass + Build Green	FASE 3	TODOS os testes passam + build + lint + deploy preview	⚡ CI	AUTO
G4	Verification Passed	FASE 4	QA confirma suite no deploy preview, 0 bugs críticos	🧪 QA	AUTO
G5	Deploy Approved	FASE 5	Homologação manual no deploy preview	👑 PO	MANUAL

+ **G6 (Process Improved):** Retrospective concluída, melhoria documentada. Liberado pelo 🎭 Maestro.

Maestro · Agile SDD com TDD

Por Que G2 é "Tests Authored (RED)"?

G2 não verifica código. Verifica TESTES.

O G2 é um gate **anterior à implementação**. O QA entrega:

- Testes E2E para 100% dos cenários Gherkin
- Testes unitários para lógica crítica
- Todos os testes **FALHAM** — e isso é bom

Por que a falha é necessária?

- Prova que o teste é válido (testa algo real)
- Se um teste passar antes do código existir, ele está errado
- A falha é a prova de que o teste vai detectar quando a feature estiver OK

Exemplo de relatório G2

```
✎ G2 Report – M01 Fundação

Status: ● RED (esperado)
Total tests: 48
Passing: 0
Failing: 48 ✔ (todos falham apropriadamente)

Cobertura Gherkin: 22/22 cenários (100%)
E2E specs: 6 (1 por feature)
Unit specs: 4 (lib, hooks, actions)
Viewports: 375px, 768px, 1280px

data-testid documentados: 34
Page Objects: HomePage, LayoutPage

✔ G2 APROVADO – testes prontos para FASE 3
```

FASE 0 — Specification

O Business Analyst transforma milestones em Gherkin

Entrada: Milestone do PRD

Saída: `specs/MXX-nome.spec.md`

Gate: 🚧 G0 — Product Owner aprova

```
## FEATURE 4: Tema Visual Base
**Como** visitante da loja
**Quero** ver uma loja com identidade visual profissional
**Para** confiar na qualidade da plataforma

### Cenário 4.1: Tema segue o sistema (happy path)
DADO que o visitante acessa a loja pela primeira vez
E o sistema operacional está configurado com tema escuro
QUANDO a página for carregada
ENTÃO o tema escuro deve ser aplicado automaticamente
E deve existir um botão de toggle para alternar para tema claro

### Cenário 4.2: Alternância manual (edge case)
DADO que o visitante está no tema escuro
QUANDO o toggle de tema for clicado
ENTÃO o tema deve alternar para claro em menos de 100ms
E não deve haver flicker visual durante a transição
```

“

💡 **Gherkin = contrato testável.** Tudo que está aqui **SERÁ** testado na FASE 2. ”

FASE 1 — Derivation

O Dev DERIVA arquitetura da spec. Sem código ainda.

Processo:

1. Dev lê cada cenário Gherkin
2. Para cada feature, projeta:
 - Schema Prisma
 - Estrutura de rotas
 - Árvore de componentes
 - Fluxo de dados
3. Documenta com rastreabilidade
4. PO aprova G1

Exemplo de rastreabilidade:

```
## Decisão: Tabela Product

### Atende aos cenários:
- 6.3: Grid de produtos (name, price, images)
  - 2.1: Listagem com filtros (categoryId)
    - 2.2: Detalhe (description)

### Schema:
- id, name, slug, description
  - price: Decimal
- images: String[] (Blob URLs)
  - categoryId: FK → Category
```

Gate G1: PO aprova arquitetura. Nenhum código foi escrito ainda.

(TDD: RED)

⚠ O QA escreve testes. ZERO código de aplicação.

Processo:

1. QA lê spec Gherkin + arquitetura
2. Para CADA cenário, escreve teste
3. Executa →  FALHA (esperado!)
4. Documenta `data-testid` esperados
5. Garante 100% cobertura

Entregáveis:

- `e2e/specs/*.spec.ts` (Playwright)
- `tests/unit/*.test.ts` (Vitest)
- Lista de `data-testid`
- Relatório RED

Exemplo de teste (escrito ANTES do código):

```
// e2e/specs/home.spec.ts
// Este teste FALHA agora - a Home Page
// ainda não foi implementada!

test('Cenário 6.1: Hero Section', async ({ page }) => {
  const home = new HomePage(page);
  await home.goto();

  //  RED: elementos não existem ainda
  await expect(home.heroTitle).toBeVisible();
  await expect(home.ctaButton)
    .toHaveText('Ver Produtos');
});
```

(TDD: GREEN + REFACTOR)

O Dev recebe testes FALHANDO e faz eles passarem

TDD Loop (por cenário):

1. Pegue 1 cenário → veja o teste ●
2. Escreva **código mínimo** para passar
3. Execute → ● GREEN? Avança. ●? Corrige.
4. ● REFACTOR: otimize mantendo ●
5. Próximo cenário

Regras:

- Nunca escreva código sem teste falhando
- Código mínimo = YAGNI (You Aren't Gonna Need It)

Após todos os cenários:

```
npm run build      # deve passar
npm run lint       # deve passar
                  # Deploy na Vercel
```

Padrões:

- ✓ TypeScript strict, nunca **any**
- ✓ **data-testid** nos elementos
- ✓ Responsivo (375, 768, 1280)
- ✓ Server Components por padrão
- ✓ **loading.tsx** + **error.tsx**

TDD na Prática — Exemplo de Sessão

QA (FASE 2): "Maestro, G2 pronto. 48 testes escritos. Todos FALHAM 🚫. Cobertura: 22/22 cenários Gherkin."

Maestro: "G2 ✅ Tests Authored. Dev, receba os testes. Eles estão em RED. Faça-os passar."

Dev (FASE 3): "Cenário 6.1: Hero Section. Teste espera `heroTitle` visível. Vou criar o componente Hero com `data-testid='hero-title'`."

Dev: "Executei o teste do Cenário 6.1. 🟢 GREEN. Refatorei o CSS. Continuo 🟢. Próximo cenário."

Dev: "Maestro, 48/48 testes passando 🟢. Build verde. Deploy preview: <https://m01.vercel.app>. G3 pronto."

Maestro: "G3 ✅ All Tests Pass. QA, FASE 4: confirme no deploy preview."

FASE 4 — Verification

QA confirma que deploy preview passa em tudo

QA re-executa:

1. Suite E2E no deploy preview
2. Smoke test manual rápido
3. Screenshots de regressão visual
4. Lighthouse audit

Classificação de bugs:

Severidade	Critério	Bloqueia?
● Crítica	Feature quebrada	SIM
● Alta	Falha grave	SIM
● Média	Limitação	NÃO
● Baixa	Cosmético	NÃO

Checklist G4:

- [] Suite E2E 100% no deploy preview
- [] Smoke test sem surpresas
- [] Sem regressão visual
- [] Lighthouse ≥ 80 / ≥ 90
- [] 0 bugs ● ou ● abertos

Se houver bugs:

- Maestro aciona Dev (FASE 3)
- Dev corrige (TDD: teste já existe!)
- QA re-verifica

FASE 5 — Acceptance

Product Owner homologa no deploy preview real

Roteiro de homologação:

1. 🎭 Maestro fornece URL do deploy preview
2. 👑 PO testa fluxos principais
3. 👑 PO verifica responsividade
4. 👑 PO faz teste exploratório
5. 👑 PO decide

✅ APROVADO

Milestone concluído! Maestro marca como CONCLUÍDO e prepara próximo.

🔧 AJUSTES

Correções pontuais. Volta à FASE 3. Se forem features NOVAS, é fura-fila.

FASE 0 — Retrospective

O framework melhora a cada ciclo

Maestro coleta feedback de todos:

-  BA: A spec estava clara?
-  QA: Os testes capturaram bem os requisitos?
-  Dev: O TDD fluiu bem? Houve retrabalho?
-  PO: Os gates fizeram sentido?

Maestro identifica ≥ 1 melhoria concreta e atualiza:

- `docs/SDLC.md` → nova versão
- `SOURCE_OF_TRUTH.md` → se necessário
- `PRD/01-milestones.md` → milestone 
- `memory/session-latest.md` → próximo

O Maestro DETECTA. Você não precisa avisar.

⚠️ **O Product Owner não precisa conhecer o termo "fura-fila".** O Maestro identifica proativamente quando algo está fora do milestone e age.

Gatilhos de detecção:

- Feature não listada no milestone atual
- "Já que estamos aqui, faz X"
- "Seria bom ter Y"
- Qualquer adição fora da spec Gherkin aprovada

Protocolo:

1. 🎭 **DETECTA** e **SINALIZA** imediatamente
2. 📋 BA analisa impacto (esforço, atraso, risco)
3. 🏰 PO decide: **A)** Fura-fila agora **B)** Próximo milestone **C)** Backlog
4. Se aprovado: milestone atual pausa, fura-fila executa FASES 0→6, milestone retoma

Definition of Done

Um milestone só está **PRONTO** quando **TODOS** os itens são verdadeiros.

#	Critério	Gate
1	Especificação Gherkin aprovada pelo Product Owner	G0 
2	Arquitetura aprovada pelo Product Owner	G1 
3	Testes E2E+unitários escritos, cobrem 100% spec, e FALHAM (RED)	G2 
4	TODOS os testes passam + build + lint verdes + deploy preview	G3 
5	QA confirmou deploy preview, 0 bugs críticos/altos	G4 
6	Product Owner homologou e aprovou	G5 
7	Retrospective concluída, SDLC e SoT atualizados	G6 
8	Código limpo: sem any , sem secrets, sem debug	—
9	Responsivo: 375px, 768px, 1280px	—
10	Lighthouse: Performance ≥ 80 , Accessibility ≥ 90	—

Estrutura de Arquivos



Exemplos de Interação

Aprovando gates

👑 "Maestro, spec aprovada. Avança."

🎭 "G0 ✅ Spec Approved. Acionando Dev para FASE 1."

Solicitando ajustes

👑 "Adiciona cenário de erro na Feature 2: banco offline."

🎭 "Ajuste registrado. BA, refinar spec M01 com cenário de erro. Re-submeto G0."

Maestro bloqueia avanço prematuro

👑 "Já manda o Dev codificar."

🎭 "Não posso. 🚧 G2 pendente. QA ainda não escreveu os testes. Testes primeiro, código depois. FASE 2 → G2 → depois FASE 3."

Fura-fila detectado

👑 "Aproveita e adiciona Página Sobre."

🎭 "⚠️ Fura-fila. M01 não inclui Sobre. +1 dia. Opções: A) Agora B) Próximo C) Backlog?"

Exemplo de Sessão Completa (TDD)

```
👑 "Maestro, começar M01."

🧑‍💻 "📄 M01 | FASE 0 | 🚧 G0 pendente.
Spec tem 22 cenários Gherkin. Aprova?"

👑 "Aprovado."

🧑‍💻 "G0 ✅. FASE 1: Dev derivando arquitetura."

[1 iteração depois]

🧑‍💻 "G1 ✅. Arquitetura aprovada. FASE 2: QA escrevendo testes."

[1 iteração depois]

🧑‍💻 "🔧 QA: 48 testes escritos. Todos 🔴 RED.
Cobertura: 22/22 cenários Gherkin. G2 pronto."

🧑‍💻 "G2 ✅. FASE 3: Dev, testes estão em RED. Faça-os passar."

[2 iterações depois]

🧑‍💻 "📄 Dev: 48/48 🟢 GREEN. Build verde. Deploy preview online."

🧑‍💻 "G3 ✅. FASE 4: QA, confirme no deploy preview."

🧑‍💻 "🔧 QA: Suíte 100% OK. 0 bugs. G4 ✅."

🧑‍💻 "FASE 5: Deploy preview https://m01.vercel.app. Homologa?"
```

O Que o Maestro NÃO Deixa Acontecer

❌ Bloqueios ativos

- Pular uma fase do SDD
- Implementar sem spec Gherkin (G0)
- Codificar antes dos testes (G2)
- Avançar com testes falhando
- Fazer deploy com bug crítico
- Tratar fura-fila como "ajuste"
- Aprovar gate sem PO explícito
- Perder contexto entre sessões

✅ Garantias

- Toda feature nasce na spec
- Testes ANTES do código
- Código mínimo (YAGNI)
- Product Owner aprova cada gate manual
- Build e lint sempre verdes
- Deploy preview para homologação
- Memória entre sessões
- Rastreabilidade total

Adaptando Para Qualquer Projeto

O Maestro Agile SDD com TDD é **agnóstico a stack e domínio**.

O que customizar vs. O que é universal

Customizável por projeto	Universal (nunca muda)
Stack (Next.js → Django, Flutter, etc.)	5 valores e 7 princípios
Agentes (Dev pode ser iOS, Data, etc.)	TDD: Testes antes do código
Milestones (domínio do negócio)	6 Quality Gates (G0-G5)
Checklists internos dos gates	Human-in-the-Loop
Nomes de pastas e arquivos	Product Owner ↔ Maestro ↔ Agentes
	Memória de sessão

O Framework em Números

5

Valores
imutáveis

7

Princípios
de governança

6

Quality Gates
bloqueantes

5

Papéis
especializados

6

Fases
do ciclo SDD

10

Critérios
no DoD

● RED → ● GREEN → ● REFACTOR · MVP em 4 sprints · Qualidade garantida

Glossário

Termo	Significado
Maestro	Agente IA central que orchestra o ciclo SDD
SDD	Specification-Driven Development — a spec é o artefato central
TDD	Test-Driven Development — testes escritos ANTES do código
RED	● FASE 2: QA escreve testes que FALHAM (código não existe)
GREEN	● FASE 3: Dev escreve código mínimo que faz o teste passar
REFACTOR	● FASE 3: Dev otimiza código mantendo testes verdes
Gherkin	Linguagem DADO / QUANDO / ENTÃO para especificações testáveis
HITL	Human-in-the-Loop — aprovação humana obrigatória em gates manuais
Quality Gate	Ponto de verificação bloqueante entre fases
YAGNI	You Aren't Gonna Need It — princípio TDD de código mínimo
Spec	Especificação formal em Gherkin — o contrato
DoD	Definition of Done — 10 critérios para milestone completo
SoT	Maestro · Agile SDD com TDD Source of Truth — documento canônico do projeto

Maestro

Agile SDD com TDD

Spec é o contrato. Testes são a prova.

● RED → ● GREEN → ● REFACTOR

Você é a aprovação.

Framework completo em `SOURCE_OF_TRUTH.md`